

Simulador y Solucionador de Cubo Rubik en Python

PARTICIPANTES:

- Leandro Bacca
- Juan Tinoco
- Sofia Mora,
- Tania Rueda

Con el siguiente informe pasaremos a explicar el funcionamiento de un programa en Python capaz de simular y resolver un Cubo Rubik 3x3 utilizando el algoritmo BFS (Breadth First Search o Búsqueda en Anchura).

1. Introducción

Este programa fue desarrollado con el objetivo de representar y simular el funcionamiento de un Cubo Rubik utilizando estructuras de datos en Python. Para esto, se creó una representación del cubo mediante matrices y diccionarios, permitiendo almacenar los colores y posiciones de cada una de sus caras. Además, el programa puede realizar los movimientos básicos del cubo, simulando de manera similar los giros que se hacen en un Cubo Rubik real.

Otra de las funciones más importantes del proyecto es que el sistema puede resolver mezclas simples automáticamente usando un algoritmo de búsqueda llamado BFS (Breadth First Search o Búsqueda en Anchura). Este algoritmo explora diferentes estados posibles del cubo hasta encontrar una solución válida. Durante el desarrollo del proyecto también se implementaron funciones para validar el cubo, mostrarlo visualmente en consola y animar paso a paso la solución encontrada.

2. Importaciones utilizadas

El programa utiliza las librerías `collections`, `copy` y `time` para diferentes funciones importantes dentro del proyecto. La librería `collections` se usa mediante `deque`, lo que permite manejar de manera eficiente la cola utilizada en el algoritmo BFS para buscar soluciones del cubo. Por otro lado, `deepcopy` de la librería `copy` permite crear copias completas del estado del cubo sin alterar el original, algo fundamental durante la exploración de movimientos. Finalmente, la librería `time` se utiliza para generar pausas

en la animación de la solución, permitiendo mostrar paso a paso cómo el Cubo Rubik se resuelve automáticamente.

3. Representación del cubo

El cubo se representa mediante un diccionario llamado estado. Cada cara del cubo es una matriz de 3x3 que contiene los colores correspondientes. Las caras utilizadas son U (arriba), D (abajo), F (frontal), B (trasera), R (derecha) y L (izquierda).

```
class Cubo:

    def __init__(self):
        self.estado = {
            'U': [['W'] * 3 for _ in range(3)],
            'D': [['Y'] * 3 for _ in range(3)],
            'F': [['G'] * 3 for _ in range(3)],
            'B': [['B'] * 3 for _ in range(3)],
            'R': [['R'] * 3 for _ in range(3)],
            'L': [['O'] * 3 for _ in range(3)],
        }

    def clonar(self):
        nuevo = Cubo()
        nuevo.estado = deepcopy(self.estado)
        return nuevo

    def como_tupla(self):
        partes = []

        for cara in ('U', 'R', 'F', 'D', 'L', 'B'):
            for fila in self.estado[cara]:
                partes.extend(fila)

        return tuple(partes)

    def resuelto(self):
        for matriz in self.estado.values():
            color = matriz[0][0]

            for fila in matriz:
                for celda in fila:
                    if celda != color:
                        return False
```

4. Movimientos del cubo

El programa implementa los movimientos básicos del Cubo Rubik: U, D, L, R, F y B. Cada función rota una cara y mueve las piezas laterales correspondientes, simulando el comportamiento de un cubo real.

5. Método aplicar()

La función aplicar() recibe movimientos como R, R' o R2. Dependiendo del sufijo, realiza giros normales, inversos o dobles.

6. Validación del cubo

La función validar_cubo() verifica que el cubo ingresado sea correcto. Para esto, comprueba que cada color aparezca exactamente nueve veces, asegurando que la configuración del cubo sea válida antes de comenzar la búsqueda de la solución.

```
def validar_cubo(cubo):  
  
    conteo = {}  
  
    for cara in cubo.estado.values():  
        for fila in cara:  
            for color in fila:  
                conteo[color] = conteo.get(color, 0) + 1  
  
    for color in COLORES:  
        if conteo.get(color, 0) != 9:  
            return False  
  
    return True
```

7. Prueba de movimientos

La función probar_movimientos() tiene como objetivo comprobar que todos los movimientos del Cubo Rubik estén correctamente programados. Para esto, el programa realiza diferentes pruebas, como aplicar un mismo movimiento cuatro veces seguidas y verificar que el cubo regrese a su estado inicial. Además, también comprueba que un movimiento y su inverso se cancelen entre sí, garantizando que la simulación funcione de manera correcta y similar a un Cubo Rubik real.

```

def probar_movimientos():

    print("\nProbando movimientos...")

    movimientos = ['U', 'D', 'L', 'R', 'F', 'B']

    for mov in movimientos:

        c = Cubo()

        original = c.como_tupla()

        for _ in range(4):
            c.aplicar(mov)

        if c.como_tupla() != original:
            print(f"X Error en movimiento {mov}")
            return False

        c = Cubo()

        c.aplicar(mov)
        c.aplicar(mov + "'")

        if c.como_tupla() != original:
            print(f"X Error en inverso {mov}")
            return False

    print("✓ Todos los movimientos funcionan correctamente.")
    return True

```

8. Algoritmo BFS

El algoritmo Breadth First Search explora todos los estados posibles del cubo por niveles. Utiliza una cola para almacenar estados pendientes y un conjunto de visitados para evitar repetir configuraciones.

9. Funcionamiento de BFS en el programa

El algoritmo comienza desde el estado inicial del cubo. Luego genera nuevos estados aplicando todos los movimientos posibles. Si encuentra el estado resuelto, devuelve la secuencia de movimientos necesaria para solucionarlo.

10. Animación de la solución

La función `animar_solucion ()` muestra el cubo paso a paso mientras se aplican los movimientos encontrados por BFS.

11. Menú principal

La función `main()` controla el flujo general del programa. Permite ingresar un cubo manualmente, ejecutar una demostración automática o salir del programa.

12. Ventajas y desventajas

La principal ventaja de BFS es que garantiza encontrar la solución más corta. Sin embargo, consume mucha memoria y tiempo cuando la profundidad aumenta.

13. Posibles mejoras

El programa podría mejorarse implementando algoritmos más eficientes como A*, heurísticas avanzadas o incluso una interfaz gráfica.

14. Conclusión

Con el desarrollo de este proyecto fue posible comprender cómo representar un problema complejo, como el Cubo Rubik, mediante programación en Python y estructuras de datos. A lo largo del trabajo se implementaron diferentes funciones que permiten simular los movimientos reales del cubo, validar configuraciones y encontrar soluciones utilizando el algoritmo BFS. Además, se logró visualizar el proceso de resolución paso a paso, haciendo que el funcionamiento del programa fuera más fácil de entender.

Este proyecto también permitió aplicar conocimientos relacionados con algoritmos de búsqueda, manejo de matrices, diccionarios y optimización de procesos. De igual manera, ayudó a fortalecer habilidades de lógica de programación y resolución de problemas, demostrando cómo la informática puede utilizarse para modelar situaciones reales y automatizar tareas complejas. Finalmente, el trabajo mostró la importancia de los algoritmos en el desarrollo de soluciones inteligentes y eficientes dentro del área de la programación.

Fragmento representativo del código

```
def resolver_bfs(cubo_inicio, limite=5):  
  
    if cubo_inicio.resuelto():  
        return [], 0  
  
    frontera = deque([(cubo_inicio.clonar(), [], None)])  
    visitados = {cubo_inicio.como_tupla()}  
  
    while frontera:  
  
        actual, camino, ultimo = frontera.popleft()  
  
        if actual.resuelto():  
            return camino
```

Este fragmento implementa el algoritmo BFS, que busca la solución del Cubo Rubik explorando estados por niveles, primero verifica si el cubo ya está resuelto. Luego utiliza una cola llamada `frontera` para guardar estados pendientes por explorar, cada vez que el algoritmo toma un estado, revisa si está resuelto. Si no lo está, genera nuevos estados aplicando movimientos. Además, usa un conjunto llamado `visitados` para evitar repetir configuraciones y optimizar la búsqueda, la ventaja principal de BFS es que encuentra la solución más corta posible.

Codigo Collab:

<https://colab.research.google.com/notebooks/intro.ipynb>